

دليل Burp Suite الشامل

من الصفر للاحتراف: التثبيت، الإعداد، وكل أدوات الأداة شرح مرتب ومتربط

· Proxy · Target · Repeater · Intruder · Decoder · Comparer · Sequencer · Logger
Organizer · Extensions

Educational / Ethical Hacking Purposes Only

إعداد: شرح مبني على محتوى غرف TryHackMe الرسمية وفيديوهات شرح عملية — بالتوفيق في رحلتك في Web Application
Penetration Testing

1. تثبيت Burp Suite وربطه بمتصفح Firefox خطوة بخطوة
2. Dashboard — الشاشة الرئيسية
3. فكرة ال Proxy — إزاي Burp يشتغل كوسيط بينك وبين الموقع
4. Target — فهم نطاق الاستهداف (Scope)
5. Intercept — Proxy و HTTP History والإعدادات المتقدمة
6. Repeater — التعديل والإرسال اليدوي
7. Intruder — الأتمتة وال Fuzzing (أنواع الهجوم وأنواع ال Payloads و Grep Match/Extract)
8. Decoder — الترميز وفك الترميز والتجزئة (Hashing)
9. Comparer — مقارنة الاستجابات
10. Sequencer — تحليل عشوائية التوكّات (Entropy)
11. Logger — سجلّ كل حركة مرور
12. Organizer — أرشفة الطلبات
13. Extensions — توسعة قدرات Burp عبر BApp Store
14. جدول مقارنة شامل بين كل الأدوات
15. كيف تترابط الأدوات مع بعضها في سيناريو اختراق حقيقي (Workflow)

1 تثبيت Burp Suite وإعداداه مع Firefox

ما هو Burp Suite أصلاً؟

لما تفتح أي موقع وتعمل Login أو تعمل Upload ملف، كل البيانات التي تتبعها بتروح للسيرفر على طول وانت ما بتشوفش الخطوات التي تحصل بالظبط من ورا الكواليس. المتصفح نفسه مايوريكش التفاصيل دي. المشكلة دي بالظبط هي التي Burp Suite جاي يحلها.

فكر في الموضوع كده: عندك جهازك، وعندك الموقع الذي بتصفحه (أو بتختبره). الحركة (Traffic) ما بين الاثنين دي بتعدي من نص — ولو حطيت Burp في النص ده، هيبقى عندك القدرة إنك:

- توقف (Intercept) أي طلب قبل ما يوصل للسيرفر وتعديل فيه.
- تحلل الاستجابات (Responses) وتفهم بنية التطبيق من جواه.
- تعيد إرسال نفس الطلب بتعديلات مختلفة آلاف المرات (Fuzzing / Brute-force).

بمنطق ال Attacker: Burp هو العدسة التي بتشوف بيها كل حركة بين ال Client وال Server، وتقدر تتحكم فيها بالكامل قبل ما توصل لوجهتها.

خطوات التثبيت

1. تحميل الأداة: من الموقع الرسمي portswigger.net/burp ، اختار نسخة Community Edition (مجانية) أو Professional لو متاحة ليك.
2. التثبيت: شغل ملف التثبيت واتبع الخطوات الافتراضية — Next → Next → اختار مكان التثبيت → Finish.

3. أول تشغيل: اختار *Temporary project* (للتجربة السريعة) أو *New project on disk* لو عاوز تحفظ شغلك، وبعدين اختار *Use Burp defaults* للإعدادات.

ربط Burp بمتصفح Firefox

بمجرد ما Burp يفتح، لازم نربطه بالمتصفح عشان كل حركة المرور تعدي عليه. أسهل وأنصف طريقة هي إضافة **FoxyProxy**:

الخطوة 1: تثبيت FoxyProxy

1. افتح Firefox، روح لـ **Settings → Add-ons and themes** (أو من متجر الإضافات مباشرة).
2. ابحث عن **FoxyProxy** — هتلاقي نسخة *Basic* ونسخة *Standard*، الاتنين بيأدوا نفس الغرض هنا، اختار أي واحدة ودوس **Add to Firefox**.
3. بعد التثبيت هتظهرلك أيقونة الإضافة الصغيرة في شريط الأدوات فوق.

الخطوة 2: إضافة بروكسي Burp داخل FoxyProxy

1. دوس على أيقونة FoxyProxy واختار **Options**.
2. دوس **Add** لإضافة بروكسي جديد واملأ البيانات التالية:

```
Title: Burp Suite
Proxy Type: HTTP
IP/Host: 127.0.0.1
Port: 8080
```

3. احفظ (Save)، وبعدين دوس على أيقونة FoxyProxy واختار بروفایل Burp اللي عملته عشان تفعّله.
- ارجع لـ "No Proxy" أو الاتصال العادي وقت ما تخلص اختبار.

الخطوة 3 (بديلة): الإعداد اليدوي من إعدادات Firefox بدون إضافة

```
Settings → General → Network Settings → Settings
اختار: Manual proxy configuration
HTTP Proxy: 127.0.0.1    Port: 8080
Use this proxy server for all protocols
```

تأكيد الإعداد من جهة Burp

افتح Burp وروح لـ **Proxy → Proxy settings** ، هتلاقي إنه افتراضياً شغال على **Loopback interface** (يعني جهازك بس) وعلى **Port 8080** — ده بالظبط اللي كتبناه في FoxyProxy. لو البورت ده مشغول عندك بحاجة تانية، تقدر تغيره من هنا، بس لازم تغير نفس الرقم في FoxyProxy كمان عشان يفضلوا متوافقين.

ثبيت شهادة (CA Certificate) Burp — ضروري لـ HTTPS

دلوقتي لو فعلت البروكسي وجربت تفتح أي موقع HTTPS، هيظهرلك تحذير أمان (Certificate warning) — وده طبيعي ومتوقع، مش عيب في الإعداد. السبب إن Burp بيعمل Man-in-the-Middle بينك وبين السيرفر: هو بيبنّي شهادة SSL خاصة بيه ويقدمها للمتصفح بدل شهادة الموقع الأصلية، عشان يقدر يفك تشفير الترافيك ويوريك المحتوى الحقيقي. والمتصفح طبعاً ييشك في شهادة Burp لأنها مش شهادة معروفة (Trusted) ليه.

الحل إنك تثبت شهادة Burp نفسها كـ Trusted CA مرة واحدة بس:

1. مع تفعيل البروكسي، افتح تاب جديد في Firefox واكتب: **http://burp**

2. دوس على **CA Certificate** ونزل الملف (**cacert.der**).

3. روح لـ **Settings → Privacy & Security → Certificates → View**

Certificates → Authorities → Import واختار الملف اللي نزلته.

4. فعل خيار "Trust this CA to identify websites" ودوس OK.

بعد الخطوة دي، جرب تفتح نفس الموقع تاني — هتلاقيه بيفتح عادي من غير أي تحذير، لأن المتصفح بقى يثق في شهادة Burp، وBurp بقى يقدر يفك تشفير ال HTTPS ويوريك ال Request/Response الحقيقي بالكامل.

فتح واجهة Burp الأساسية

بعد التثبيت هتلاقي الواجهة الرئيسية مقسمة لتابات (Tabs) فوق، وكل تاب ده أداة مستقلة بنفسها:

، Sequencer ، Repeater ، Intruder ، Proxy ، Target ، Dashboard ، Decoder ، Comparer ، Logger ، Organizer ، Extensions . باقي الكتاب هيشركك كل تاب فيهم بالترتيب اللي غالباً هتستخدمهم بيه فعلياً.

Dashboard 2 — الشاشة الرئيسية

الفكرة Dashboard هو أول تاب بتشوفه لما تفتح Burp، وهو بمثابة "غرفة التحكم" — بيدّيك نظرة سريعة على كل حاجة شغالة أو اتكشفت لحد دلوقتي في المشروع، من غير ما تضطر تدور في كل تاب لوحده.

أقسام Dashboard

القسم	وظيفته
Tasks	قائمة بالمهام الشغالة أو المجدولة — زي Live passive crawl اللي يشتغل تلقائي في الخلفية وهو يحلل كل حركة مرور بتعدّي على ال Proxy
Issue activity	الثغرات (Issues) اللي Burp اكتشفها تلقائياً أثناء تصفحك، مرتبة حسب الخطورة (Severity): High / Medium / Low / Information
Advisory	لما تدوس على أي Issue من القائمة، يظهر لك شرح تفصيلي عن الثغرة، ليه هي خطيرة، وإزاي تتأكد منها أو تصلحها (مرجع ممتاز حتى لو بتعلم Web Security من الصفر)
Event log	سجلّ زمني لكل حاجة Burp عملها من لحظة فتح المشروع — مفيد للتتبع ومعرفة تاريخ الجلسة

في Burp Community Edition، معظم ال Issues اللي بتظهر هنا بتيجي من ال **Passive scanning** (تحليل سلبي لحركة المرور اللي أنت بالفعل بتعملها أثناء تصفحك العادي)، لأن ال **Active scanning** الكامل (فحص الموقع آلياً وبكثافة) ميزة حصريّة في Burp Professional.

هل Dashboard مهم فعلياً؟

مش هتستخدمه بنفس كثافة ال Proxy أو ال Repeater، لكنه مكان ممتاز تبدأ بيه كل جلسة اختبار: شوف إيه اللي Burp لقاه تلقائي الأول (ده بيوفّر عليك وقت)، وبعدين كّل بالتحليل اليدوي العميق في باقي الأدوات.

3 فكرة ال Proxy بشكل عام

المسار: Browser ⇌ (127.0.0.1:8080 على Burp Proxy) ⇌ Server

أي حركة منك بتعدّي على Burp الأول، وهو اللي يحللها قبل ما يبعثها للموقع، والرد كمان يرجع من خلاله قبل ما يوصلك. وبالتالي كل الأدوات الثانية في (Burp (Repeater, Intruder, Sequencer... بتأخذ الطلبات اللي اتمسكت في ال Proxy module كنقطة بداية — يعني ال Proxy هو "قلب" الأداة، وكل حاجة بعد كده مبنية عليه.

من منظور Attacker vs Defender:

• **Attacker**: يستخدم ال Proxy عشان يفهم بنية التطبيق، يلاقي Parameters مخفية، ويعدّل بيانات قبل ما توصل السيرفر.

• **Defender**: لازم يتأكد إن أي تحقق (Validation) يحصل في ال Server-side مش ال Client-side بس، لأن ال Proxy بيثبت إن أي حاجة في المتصفح ممكن نتعدل بسهولة.

Target 4 — فهم نطاق الاستهداف (Scope)

الفكرة أول ما تبدأ نتصفح من خلال Burp، تاب ال Target يبين لك خريطة (Site map) لكل حاجة مرّت من خلاله — حتى لو مالهش علاقة بالموقع اللي انت فاكر إنك بتتصفحه بس.

ليه بتظهرلي دومينات مش متوقعة؟

لو فتحت مثلاً `facebook.com` بس، هتلاقي في ال Target حاجات كتير قوي ظهرت معاها — زي `mozilla.com` مثلاً — رغم إنك ماخلتش عليها خالص. السبب إن المتصفح والموقع بيعملوا طلبات خلفية كتير (تتبع، إحصائيات، خدمات خارجية...) من غير ما تحس. المشكلة إن ده بيعمل "ضوضاء" (Noise) بتوهك عن الهدف الحقيقي بتاعك.

تحديد ال Scope

1. من تاب Target، روح ل `Scope settings`.
2. دوس `Add` وحدد الدومين اللي انت فعلاً مستهدفه، مثلاً: `facebook.com`. أي Subdomain تابع له (زي `api.facebook.com` أو أي حاجة خاضعة له) هيتضاف تلقائياً ضمن النطاق.
3. في خانة `Exclude from scope` تقدر تستبعد أي دومين بتلاحظ إنه بيظهر كتير وملوش علاقة بهدفك (زي `mozilla.com` في المثال اللي فوق).

فلتر العرض حسب ال Scope

بعد ما تحدد ال Scope، دوس على أيقونة ال Filter في أعلى الجدول، وفعل خيار `Show only in-scope items`. دلوقتي ال Target هيبورك بس الحاجات اللي فعلاً متعلقة بهدفك. نفس فكرة الفلتر دي موجودة كمان في HTTP History داخل Proxy، وتقدر تستخدمها هناك بنفس الطريقة.

تقدر كمان تفلتر حسب نوع الملف (زي إظهار JS و CSS بس، أو إخفاءهم) عشان تركز على ال Requests المهمة فعلاً وتبعد عن تفاصيل العرض (Static assets) اللي مش هتفيدك في الاختبار.

5 Intercept — Proxy، ال History، والإعدادات المتقدمة

الفكرة تاب ال Proxy هو الأهم على الإطلاق في Burp، لأنه هو اللي يتحكم فعلياً في محتوى أي صفحة بتحملها — تقدر توقف أي طلب، تشوف محتواه، تعدّله، أو حتى توقفه نهائياً قبل ما يكل.

Intercept — إيقاف الطلبات يدوياً

لما يكون زرار **Intercept is on** شغال، أي طلب هيجتاج منك تدوس **Forward** يدوياً عشان يكل مشواره للسيرفر، أو **Drop** عشان توقفه نهائياً. المشكلة إن الصفحة الواحدة النهاردة بتعمل عشرات الطلبات (HTML، CSS، JS، صور، خطوط...)، فلو سبت Intercept شغال وانت بتتصفح بشكل عادي، هتلاقي نفسك بتدوس Forward عشرات المرات لصفحة واحدة بس.

عشان كده: سيب Intercept شغال بس وقت ما انت فعلاً محتاج توقف طلب معين (زي طلب Login أو Upload)، واقفله (Off) وانت بتتصفح بشكل عادي — Burp هيفضل يسجل كل حاجة في HTTP History في الخلفية برضو.

لو قررت أثناء التصفح إنك مش عايز طلب معين يكل أصلاً (مثلاً دخلت غلط على موقع مش هدفك)، تقدر تدوس **Drop** عليه بدل Forward، وهيتوقف فوراً من غير ما يوصل للسيرفر.

HTTP History — سجل كل حاجة مرّات

سواء Intercept شغال أو لا، كل طلب ورد بيتّرجل في تاب **HTTP History** جوه Proxy. ده بيدّيك القدرة إنك ترجع لأي طلب قديم في أي وقت وتحلله، حتى لو ما وقفتوش وقت ما حصل. زي ال Target بالظبط، تقدر تفلتر ال History بال Scope أو بنوع المحتوى (مثلاً تعرض ال JS أو HTML بس) عشان تلاقي اللي يهملك بسرعة.

WebSocket History

بعض المواقع تستخدم اتصال دائم (Persistent connection) بين المتصفح والسيرفر اسمه **WebSocket** بدل الطريقة التقليدية في كل Request/Response منفصل. Burp يسجل الاتصالات دي في تاب منفصل اسمه **WebSocket History**. الموضوع ده موضوع كبير بحاله ومحتاج شرح منفصل، بس المهم تعرف إنه موجود ولو هدفك يستخدم Real-time features (شات مباشر، إشعارات لحظية...) فرجح إنك هتلاقه هناك.

Proxy Settings — صندوق أدوات ضخمة

تاب **Proxy Settings** فيه عدد كبير من الخيارات المفيدة اللي بتوفر عليك وقت كبير في التعديل اليدوي، لأنها بتطبق تلقائياً على كل طلب أو رد بيعدي:

- **تغيير ال Listener**: تقدر تغير رقم البورت (لو 8080 مشغول عندك بحاجة تانية)، أو تخلي ال Listener يشتغل على كل ال Network interfaces بدل ال Loopback بس — ده مفيد جداً لو عايز تمرر حركة مرور جهاز تاني (موبايل مثلاً) من خلال جهازك: بتدي الموبايل ال IP بتاع جهازك ونفس البورت في إعدادات Wi-Fi بتاعه، وتفتح نفس خطوة تثبيت الشهادة على المتصفح بتاع الموبايل.
- **Unhide hidden form fields**: كشف أي Input field معمول له **hidden** في الصفحة عشان تقدر تشوفه وتعديل فيه — مفيد لأن المبرمجين أحياناً يحطوا قيم حساسة في حقول مخفية ظناً منهم إن المستخدم مش هيشوفها.
- **Remove input field length limits**: إزالة أي قيود على طول الحقل (زي **maxlength** في فورم اليوزرني أو الباسورد) عشان تقدر تختبر بقيم أطول من المسموح بيها في الواجهة — وده بيثبت إن القيود دي مجرد واجهة (Client-side) ومش حماية حقيقية.
- **Remove JavaScript validation**: إزالة أي تحقق يحصل بال JavaScript قبل الإرسال — مفيد جداً مثلاً لو فورم رفع ملفات بيرفض امتداد معين (زي **.php**) من جوه المتصفح؛ بتشيل التحقق ده وتخلي الفحص الحقيقي يحصل في السيرفر (لأن ده هو الفحص اللي فعلاً بيهم أمنياً).
- **Convert HTTPS links to HTTP**، **Remove Secure flag from cookies**: خيارات

بتساعد في اختبارات التلاعب بالبروتوكول والكوكيز.

القاعدة الذهبية هنا: أي تحقق أو قيد بتشوفه بيتشال بسهولة من Burp (زي حد أقصى لطول الباسورد، أو امتداد ملف ممنوع) يبقى معناه إنه تحقق Client-side بس — ولو مفيش نفس التحقق متكرر في السيرفر، يبقى التطبيق فيه ثغرة حقيقية.

الإعدادات دي كتير جداً ومتجددة، وده مجرد تذوق سريع ليها — انصحك تاخذ وقتك وتستكشف باقي الخيارات بنفسك وقت ما تحتاجها فعلياً.

Referer Header

ال **Referer** هو ال Header اللي يقول للسيرفر "المستخدم جاي منين" — مثلاً دخلت جوجل وبحثت عن "تويتر" ودوست على النتيجة، فال Referer هيكون صفحة نتائج البحث بتاعة جوجل. تقدر تعدّل أو تزور القيمة دي من خلال ال Proxy (أو تستخدمها لاحقاً في Repeater) عشان تختبر هل التطبيق بيعتمد عليها في أي قرار أمّني — لأن لو بيعتمد عليها، فده اعتماد خطأ لأنها قيمة سهلة التزوير.

Repeater 6 — التعديل والإرسال اليدوي

الفكرة Repeater يبدّي طلب واحد تقدر تعدّل فيه وتبعته ثاني وثاني، وتقارن كل مرة بال Response اللي راجع. ده أنسب أداة لما تكون بتختبر حاجة محددة بشكل يدوي ومركّز — زي SQL Injection أو IDOR — عكس ال Intruder اللي بيعت مئات الطلبات تلقائي.

إزاي تبعت طلب لـ Repeater؟

من ال Proxy module (أو أي تاب ثاني فيه طلب)، كليك يمين على ال Request واختار **Send to Repeater**، أو اختصار **Ctrl + R**.

أقسام واجهة Repeater

القسم	وظيفته
Request List	قائمة بكل الطلبات المحفوظة في Repeater، ممكن تدير أكثر من طلب في نفس الوقت
Request Controls	أزرار Send / Cancel والتنقل بين ال History
Request & Response View	تعديل الطلب على الشمال، ومشاهدة الرد على اليمين
Layout Options	تبديل بين عرض أفقي / رأسي / تابات منفصلة
Inspector	تحليل وتعديل الطلب بشكل أكثر سهولة (بصري) بدل التعديل في ال Raw Editor مباشرة
Target	ال IP أو ال Domain اللي الطلب رايح له

طرق عرض ال Response

- **Pretty**: الوضع الافتراضي — تنسيق يسهل القراءة.
- **Raw**: الاستجابة الخام زي ما جاية من السيرفر بدون أي تعديل.
- **Hex**: تمثيل بايتي — مفيد مع الملفات ال Binary.
- **Render**: عرض الصفحة زي ما هتظهر في المتصفح فعلياً.

Inspector في Repeater

بين لك بشكل جدولي (Tabular) أقسام الطلب: Request Attributes (method, path, protocol)، Response Headers، Cookies، Body Parameters، Query Parameters... وبعد الرد: Response Headers (للقراءة فقط، مش قابلة للتعديل لأنك مالكش تحكم في السيرفر).

مثال بسيط: تغيير Header والمقارنة بالرد

أبسط تجربة ممكن تعملها في Repeater إنك تاخد طلب عادي، تبعته (Send) وتشوف الرد، وبعدين تعدّل في حاجة بسيطة زي ال **Host** Header أو قيمة ال **Connection** وتبعت تاني — هتلاحظ إن الرد بيتغيّر على طول (ممكن تاخد خطأ Host غير معروف، أو يتغيّر سلوك الاتصال). التجربة الصغيرة دي بتوضحك إزاي السيرفر بيكون حساس لأي تعديل، وإزاي تبني عليها اختبارات أعمق.

مثال عملي: Header Manipulation لتجاوز تحقق بسيط

```
GET / HTTP/1.1
Host: MACHINE_IP
...
FlagAuthorised: True
```

لو السيرفر بيتحقق من وجود Header معين عشان يديك صلاحية إضافية، وما فيش تحقق حقيقي من هوية المستخدم، تقدر تضيف ال Header يدوياً من Inspector أو من ال Raw Request وتبعت — مثال بسيط على فكرة Trust Boundary الخاطئة.

مثال عملي: استغلال IDOR عن طريق التلاعب في رقم مسار (Path Parameter)

لو موقع فيه endpoint زي `/products/3` ، وغيرت الرقم لحاجة متطرفة (زي `-1` أو رقم ضخم جداً)، ممكن يحصل Internal Server Error 500 — ده مؤشر إن مفيش Input Validation كافي على ال Server-side، وممكن يكشفك تفاصيل داخلية عن قاعدة البيانات (Verbose Error Messages).

مثال متقدم: Union-based SQL Injection عن طريق Repeater

دي أقوى استخدامات Repeater — لإنك محتاج تبعت نفس ال Payload بتعديلات دقيقة ومتتابة:

1. **Confirm** الثغرة: ضيف علامة اقتباس ' في نهاية ال parameter — لو السيرفر رجع Error 500، معناها فيه SQL Injection محتملة.

2. استخرج اسم الجدول وعدد الأعمدة من رسالة الخطأ لو السيرفر verbose (زي `SELECT` `(firstName, lastName... FROM people WHERE id = 2'`).

3. عدّ الأعمدة واستخرج أسماءها باستخدام `UNION ALL SELECT` مع `: information_schema.columns`

```
/about/0 UNION ALL SELECT
group_concat(column_name),null,null,null,null FROM
information_schema.columns WHERE table_name="people"
```

4. اسحب البيانات المستهدفة:

```
/about/0 UNION ALL SELECT notes,null,null,null,null FROM people
WHERE id = 1
```

ليه بنستخدم `0` بدل ال ID الحقيقي؟ عشان نضمن إن ال Query الأصلية مترجعش أي صف حقيقي، فالصف الأول اللي ه يظهر في الصفحة يبقى هو ناتج ال UNION Query بتاعتنا.

الفرق الجوهرى بين Repeater و Intruder هنا: فى ال SQLi ده احنا محتاجين نفكر ونحلل كل Response قبل ما نعمل الخطوة اللى بعدها — ده استخدام يدوي تحليلي، مش أتمتة عمياء. لو كنت عاوز تجرب آلاف ال Payloads أو أسماء جداول، وقتها Intruder هو الأنسب.

Intruder 7 — الأتمتة وال Fuzzing

الفكرة Intruder هو أداة ال Fuzzing المدمجة في Burp، شبيهة بأدوات زي **ffuf** أو **wfuzz** لكن بواجهة رسومية. يباخذ Template لطلب واحد ويبيعه مئات أو آلاف المرات بقيم مختلفة (من Wordlist) في أماكن محددة.

في Burp Community Edition، Intruder يشتغل بسرعة محدودة (Rate-limited) — وهتلاقي تحذير بكدّه لما تبدأ هجوم. ده بيخلي ناس كتير تفضل أدوات تانية لل Brute-forcing الكبير، لكنه برضو أداة قوية وسهلة للمهام المتوسطة.

تأبات Intruder الأربعة

وظيفته	التاب
تحديد مكان ال Payload في الطلب (مخاط بعلامة §) واختيار نوع الهجوم	Positions
تحديد القيم اللي هتتخط في الأماكن دي (Wordlist, Number range...)	Payloads
إدارة الموارد بين المهام (مفيدة أكثر في Professional)	Resource Pool
التحكم في سلوك الهجوم (زي معالجة ال Redirects، أو flag على كلمات معينة)	Settings

Positions — تحديد نقاط الحقن

Burp يحاول يحدد الأماكن المحتملة تلقائياً ويحوطها بعلامة § (section mark). عندك ثلاث أزرار:

• **Add** §: تضيف مكان جديد يدوياً (حدد النص ودوس عليه).

• **Clear** §: تمسح كل الأماكن وتبدأ من الصفر.

• **Auto** §: ترجع الاختيار التلقائي لو مسحته.

أنواع الهجوم (Attack Types) — أهم حاجة في Intruder

Sniper

الاقتراضي والأكثر استخداماً. يستخدم Payload Set واحد بس، ويحرب كل قيمة في كل مكان على حدة (مكان واحد في كل مرة، والباقي يفضل زي ما هو من ال Original Request).

Body	#
username=burp&password=Expl01ted	1
username=suite&password=Expl01ted	2
...	...
username=pentester&password=burp	4

عدد الطلبات = عدد الكلمات × عدد الأماكن. مثال: Wordlist فيه 100 كلمة وعندك 3 مواضع → 300 طلب.

مناسب لـ: Fuzzing مكان واحد (زي endpoint IDs)، أو اختبار كل parameter لوحده بنفس ال Wordlist.

Battering Ram

يستخدم Payload Set واحد برضو، لكن يحط نفس القيمة في كل الأماكن في نفس الوقت.

Body	#
username=admin&password=admin	1
username=Guest&password=Guest	2

مناسب لـ: حالات زي لما ال username وال password نفس القيمة، أو اختبار Race Conditions.

Pitchfork

يستخدم Payload Set منفصل لكل مكان (لحد 20)، ويمشي عليهم بالتوازي — يعني ياخذ العنصر الأول من كل Wordlist مع بعض، بعدين الثاني مع الثاني، وهكذا.

#	Body
1	username=joel&password=J03l
2	username=harriet&password=Emma1815
3	username=alex&password=Sk1ll

مناسب لـ: Credential Stuffing — لما يكون عندك لسته يوزرات ومعاها الباسوورد الصح بتاع كل واحد (نفس الترتيب)، زي بيانات مسروقة من تسريب (Data Breach).

لازم كل ال Payload Sets تكون بنفس الطول، وإلا Intruder هيقف عند أقصر لسته.

Cluster Bomb

زي Pitchfork لكن يجرب كل التركيبات الممكنة بين كل ال Payload Sets (كل تركيبة مع كل تركيبة).

#	Body
1	username=joel&password=J03l
2	username=harriet&password=J03l
3	username=alex&password=J03l

...	كل تركيبة مع كل تركيبة...
-----	---------------------------

عدد الطلبات = حاصل ضرب أطوال كل الـ Lists. مثال: $100 \times 2 \times 30 = 6000$ طلب.

مناسب لـ: لما يكون عندك يوزرات وباسوردات بس مش عارف أي باسورد يخص مين (Brute-force كامل بدون معرفة الربط).

جدول مقارنة سريع بين الأربع أنواع

النوع	عدد Payload Sets	طريقة الحقن	أفضل استخدام
Sniper	1	مكان واحد في كل مرة	Fuzzing endpoint / parameter واحد
Battering Ram	1	نفس القيمة في كل الأماكن معاً	Race / Default creds متطابقة / conditions
Pitchfork	حتى 20	بالتوازي (Index متطابق)	Credential Stuffing
Cluster Bomb	حتى 20	كل التركيبات الممكنة	Brute-force كامل بدون معرفة الربط

Payloads Tab — تفاصيل أعمق

- **Payload Sets:** اختيار المكان ونوع الـ Payload.
- **Payload Settings:** إضافة/حذف القيم يدوياً، أو تحميل Wordlist كامل من ملف بزرار **Load**.
- **Payload Processing:** قواعد تنطبق على كل Payload قبل الإرسال — زي **Add prefix** / **Match & Replace** / **Add suffix** / تجاهل القيم اللي بتطابق Regex معين.

• **Payload Encoding**: افتراضياً Burp يعمل URL-encode للـ Payload؛ تقدر تعدّل القائمة أو تلغي الخاصية دي.

أنواع الـ Payload (Payload Types) — مش بس Simple List

في مثال الـ Credential Stuffing استخدمنا نوع **Simple List** (كتابة أو تحميل قائمة كلمات جاهزة)، لكن Burp فيه أنواع تانية كتير مناسبة لمواقف مختلفة:

النوع	بيعمل إيه	مثال استخدام
Simple list	قائمة كلمات ثابتة (يدوي أو من ملف)	Wordlist يوزرات أو باسوردات
Numbers	يولّد أرقام في مدى معين (From / To / Step)، بصيغة Sequential أو Random	Fuzzing لرقم ID في endpoint (زي / <code>products/1</code> لحد <code>/products/500</code>)
Dates	يولّد تواريخ في مدى زمني معين وبصيغة محددة	اختبار Endpoints بتاخذ تاريخ كـ Parameter
Brute forcer	يولّد كل التراكيب الممكنة من مجموعة حروف محددة بطول معين (مش قائمة جاهزة، بل توليد شامل)	تجربة كل الأكواد الرقمية الممكنة المكوّنة من 4 أرقام مثلاً (0000 → 9999)
Character substitution	ياخد كلمات أساسية ويولّد منها صيغ بديلة بإحلال حروف بأرقام أو رموز شبيهة (زي <code>a</code> → <code>@</code> , <code>e</code> → <code>3</code>)	تغطية الأشكال اللي اليوزر بيستخدمها عادة بدل الحرف الأصلي في الباسورد
Case modification	يولّد نسخ مختلفة من نفس الكلمة بحالات أحرف متعددة (كله كبير، كله صغير، أول حرف كبير...)	تغطية احتمالات الكتابة المختلفة للباسورد الواحد

القاعدة العملية: لو عندك قائمة جاهزة (زي باسوردات مسربة) استخدم Simple list. لو محتاج تغطي مدى أرقام أو تواريخ، استخدم Numbers أو Dates. لو عايز تجرب كل الاحتمالات الممكنة لطول معين من غير قائمة جاهزة، استخدم Brute forcer — بس احذر، العدد بيكبر بسرعة رهيبة كل ما زاد الطول.

Grep Match و Grep Extract — أتمتة قراءة النتائج

لحد دلوقتي كنا بنخلل نتايج Intruder يدويًا (نرتب بال Length، أو نفتح كل Response لوحده). في الواقع العملي ده بياخد وقت كبير خصوصًا مع آلاف الطلبات. الحل موجود في تاب Settings جوه Intruder:

• **Grep Match**: بتدي Burp كلمة أو Regex معين (زي `Invalid username` أو `Welcome`)

(`back`)، وهو يفتح كل Response ويضيف عمود جديد في الجدول يوضح هل الكلمة دي موجودة فيه ولا لا (/). ده بيديك فلترة بصرية فورية من غير ما تفتح كل رد لوحده — مثلاً تحدد Grep Match على عبارة "Invalid password" فأني رد من غيرها يبقى مرشح قوي للنجاح.

• **Grep Extract**: أقوى من كده — بيسمحك تستخرج قيمة معينة من جوه ال Response (زي CSRF Token، أو رقم معين، أو اسم ظاهر في الصفحة) وتظهرلك في عمود منفصل لكل طلب، بدل ما تفتح كل Response وتدور عليها يدويًا. بتحددها عن طريق تمييز (Highlight) النص المطلوب في رد نموذجي، وBurp يبني ال Pattern تلقائيًا.

مثال عملي: لو بتعمل Sniper على Endpoint بيرجع اسم المستخدم جوه ال HTML، تقدر تستخدم Grep Extract على مكان الاسم ده، فيبقى عندك عمود "الاسم المستخرج" لكل Request من غير فتح أي Response يدويًا — بيوفر وقت ضخم مع آلاف الطلبات.

Recursive Grep

ميزة متقدمة بتسمحك تستخدم قيمة استخرجت من Response واحد ك Input في الطلب اللي بعده مباشرة — يعني Burp بياخد نتيجة Grep Extract من الرد الحالي ويحققها تلقائيًا في الطلب التالي قبل ما يبعته. دي

مفيدة في سيناريوهات فيها قيمة متغيرة لازم تتجدد كل طلب (زي توكن مؤقت) من غير ما تحتاج تبني Macro كامل زي اللي عملناه قبل كده.

ملحوظة من الغرفة الرسمية: Recursive Grep مش بيشتغل صح مع بعض أنواع الردود زي ال Redirects (3xx) — في الحالة دي، الحل البديل هو استخدام Session Handling Rules مع Macro زي اللي شرحناه في مثال Credential Stuffing المتقدم.

مثال تطبيقي كامل: تجربة Brute-force على فورم Login

السيناريو ده بيوضح الفكرة من أول خطوة للنتيجة:

1. فعل Intercept في Proxy، وادخل على صفحة ال Login بتاعة الهدف.
2. اكتب أي يوزرنيـم وباسورد تجريبيين ودوس Burp — Login هيقف الطلب (POST request).
3. كليك يمين على الطلب واختار **Send to Intruder** (أو **Ctrl + I**).
4. في تاب Positions، امسح الأماكن التلقائية (**Clear §**) وحدد بس الحقل اللي عايز تختبره — مثلاً حقل الباسورد بس، وسيب اليوزرنيـم ثابت (ده هجوم Sniper على مكان واحد).
5. روح لتاب Payloads: اكتب الكلمات يدوياً في خانة الإضافة، أو استخدم **Load** لرفع Wordlist جاهزة.
6. دوس **Start attack**. هيطهرلك تحذير عن إن النسخة المجانية بطيئة (Rate-limited) — اضغط OK وكّل.
7. بعد ما يخلص، راجع عمود ال Response (الطول، كود الحالة) لكل محاولة عشان تكتشف هل في باسورد نجح ولا لأ.

التجربة دي بتعلّمك ال workflow الكامل حتى لو محصلش نجاح فعلي — المهم إنك نتقن الخطوات: التقاط الطلب، تحديد المكان، تجهيز ال Payloads، تشغيل الهجوم، وتحليل النتيجة.

مثال عملي متقدم: Credential Stuffing مع Macros (CSRF Token متغير)

لما يكون فورم فيه CSRF Token (`loginToken`) و Session Cookie يتغيروا مع كل Request، ال Pitchfork العادي مش هيكفي — لازم **Burp Macro** يجيب التوكن الجديد تلقائياً قبل كل طلب:

1. اعمل Macro من **Settings → Sessions → Macros** بيعت GET request لصفحة اللوجن.

2. اعمل Session Handling Rule، وحدد نطاقه على Intruder بس (Tools Scope).

3. في ال Rule Actions اختار **Run a Macro**، وحدد إنه يحدث بس ال `loginToken` parameter وال `session` cookie.

4. شغل الهجوم ب Pitchfork عادي — الماكرو هيقن قيم جديدة صحيحة قبل كل طلب تلقائياً.

لو شفت كل الردود 403 بدل 302، معناه الماكرو مش شغال صح — راجع إعدادات ال Scope وال Parameters اللي بتحدث.

تحليل النتائج: إزاي تعرف أي Request نجح؟

غالباً status codes بتكون متشابهة (زي 302 Redirect لكل المحاولات). الحل: رتب النتائج بعمود **Length** — الطلب الناجح غالباً بيكون له طول Response مختلف (أقصر عادةً) عن باقي المحاولات الفاشلة.

Decoder 8 — الترميز، فك الترميز، والتجزئة

الفكرة Decoder هو أداة تحويل بيانات — بتعمل Encode (ترميز) أو Decode (فك ترميز) أو Hash (تجزئة) لأي نص أو بيانات، زي ما تعمل في CyberChef بس بشكل مبسط جوه Burp نفسه.

أنواع الترميز/فك الترميز

النوع	الاستخدام
Plain	النص الخام بدون أي تحويل
URL	تحويل الحروف الخاصة لصيغة <code>%XX</code> هيكساديسيمال — أساسي في طلبات الويب
HTML	تحويل الحروف الخاصة لصيغة <code>&#...;</code> — يمنع مشاكل ال Rendering وهجمات XSS
Base64	تحويل أي بيانات لصيغة ASCII آمنة للنقل
ASCII Hex	تحويل بين النص العادي والتمثيل الهيكساديسيمال لكل حرف
/ Hex / Octal Binary	خاصة بالأرقام فقط — تحويل بين الأنظمة العددية
Gzip	ضغط/فك ضغط البيانات

الطرق دي قابلة لـ **Stacking** (تطبيق أكثر من طبقة فوق بعض) — مثلاً: نص → ASCII → Base64
Hex → Octal.

Smart Decode

خاصية بتحاول تكتشف نوع الترميز تلقائياً وتفك الطبقات واحدة ورا الثانية، زي خاصية "Magic" في CyberChef. مفيدة جداً لما تلاقي بيانات غريبة ومش عارف هي متشفرة بإيه.

Hex View

بين البيانات بشكل Byte-by-byte — مهم مع الملفات الـ Binary أو البيانات اللي مش نص عادي (ASCII/Unicode).

استخدام عملي: تجاوز فلتر الـ WAF

لو بتبني Payload (زي XSS مثلاً) وكل مرة Web Application Firewall بيوقفه، تقدر تستخدم Decoder عشان تجرب تشفير الـ Payload بطريقة (URL encoding مثلاً، أو مزيج من أكثر من ترميز) تخليه يعدّي من غير ما الفلتر يتعرف عليه بشكله الواضح. وبالعكس، لو لقيت قيمة غريبة في Cookie أو Parameter جايالك من الموقع، تقدر تلزقها في Decoder وتعمل Decode عشان تفهم شكلها الحقيقي.

الـ Hashing في Decoder

الـ Hashing عملية باتجاه واحد (One-way) — يعني مينفعش ترجع من الـ Hash للنص الأصلي. أي تغيير بسيط في المدخل بيغيّر الـ Hash بالكامل (Avalanche Effect).

خوارزمية MD5 قديمة (Deprecated) ومش آمنة للاستخدامات الحديثة بسبب إمكانية عمل Collisions — استخدمها بس للتحقق من سلامة ملفات (Integrity Check) مش لتخزين كلمات سر.

مبدأ تخزين كلمات السر: السيرفر مايخزنش الباسورد الأصلي أبداً — يخزن الـ Hash بتاعه، وعند تسجيل الدخول بيعمل Hash للمدخل ويقارنه بالمخزن.

استخدام هجومي شائع: لو عندك Hash مجهول (زي SSH key مش معروف مصدره)، تقدر تعمل Hash لكل الاحتمالات الممكنة وتقارن بالقيمة المعروفة لحد ما تلاقي التطابق (زي استخدام `md5sum` في Linux بدل Decoder لما يكون عندك ملفات كتير).

9 Comparer — مقارنة البيانات

الفكرة أداة بسيطة لكنها قوية جداً — بتقارن بين قطعتين بيانات (Request أو Response) وتوريك الفروق بينهم بالألوان، إما على مستوى **Words** (كلمات) أو **Bytes** (بايتات).

أقسام الواجهة

- الجزء الشمال: جدول فيه كل البيانات اللي اتحطت في Comparer (من Send to Comparer).
 - أعلى اليمين: Paste / Load / Remove / Clear.
 - أسفل اليمين: زرار Words و Bytes — تختار اثنين من الداتا وتضغط عليهم للمقارنة.
- بعد المقارنة بتفتح نافذة فيها الفروق ملونة (Modified / Deleted / Added)، وتقدر تشغل **Sync views** عشان الاتنين يتبدلوا لنفس صيغة العرض (Text أو Hex) مع بعض.

متى تستخدمه عملياً؟

أشهر استخدامين:

- بعد هجوم **Intruder**: لما ال Status Codes كلها متشابهة (302 مثلاً)، تاخذ Response طلب فاشل و Response طلب (محتمل) ناجح وتقارن بينهم ب Words عشان تكتشف الفرق الدقيق اللي يأكذك نجاح الهجوم.
- أثناء اختبار **SQL Injection**: تبعت طلبين لنفس ال Endpoint، الفرق الوحيد بينهم إنك حققت Payload في واحد منهم (زي `id=1` مقابل `id=1'` أو إضافة `UNION SELECT`)، وتقارن الاتنين. Comparer هتوريك بالظبط فين اختلفت الاستجابة — سواء محتوى اتغير، أو حتى تأخير زمني واضح في الرد (مؤشر على **SQLi** من نوع **Time-based Blind**).

Sequencer 10 — تحليل عشوائية التوكنات (Entropy)

الفكرة

Sequencer يختبر مدى عشوائية (Randomness / Entropy) أي Token — زي Session Cookie أو CSRF Token. لو التوكن مش عشوائي كفاية بشكل كريبتوجرافي، ممكن نظرياً تتنبأ بالقيم الجاية، وده خطير جداً خصوصاً لو التوكن مستخدم في عمليات زي Password Reset.

ليه العشوائية مهمة؟ مثال Session Fixation

المفروض إن كل مرة مستخدم بيعمل Login، ال Session ID / Cookie بتاعته يتغير (Regenerate) من جديد. لو التطبيق بيستخدم نفس ال Session ID الثابت وما بيغيروش بعد تسجيل الدخول، بتظهر ثغرة اسمها Session Fixation: المهاجم يقدر يجهز Session ID معروف له مسبقاً، يخلي الضحية تستخدمه (من غير ما تحس)، وبجهد ما الضحية تعمل Login، المهاجم يقدر يستخدم نفس ال Session ويدخل على حسابها مباشرة. دي ثغرة مختلفة تماماً عن CSRF، و Sequencer هو أداة اكتشافها الأساسية.

طريقتين للتحليل

- **Live Capture** (الأشيع): بتدي Burp طلب بيولد Token، وهو بيكره آلاف المرات (حوالي 10,000 عينة) ويجمع العينات للتحليل.
- **Manual Load**: ترفع لسته Tokens جاهزة اتجمعت قبل كده — مفيدة لما ميكنش عندك نية تبتع آلاف الطلبات (أقل وضوء وأسرع).

خطوات Live Capture

1. ابعت Request ل Sequencer (**Send to Sequencer**) — طلب الرد بتاعه يحتوي على التوكن اللي عايز تحلله (مثلاً رد فيه Cookie).
2. حدد مكان ال Token: Cookie / Form field / Custom location، واختار التوكن بالتحديد من القائمة.

3. دوس Start live capture — Burp هيبدأ يبعث طلبات كثير تلقائي ويجمع عينات. لازم يكون عندك على الأقل حوالي 100 عينة كحد أدنى للبداية، وكل ما زادت كل ما كان التحليل أدق (الوضع المثالي حوالي 10,000 عينة).

4. دوس Pause (مش Stop، عشان تقدر تكمل لو محتاج عينات أكثر) ثم Analyze now.

قراءة التقرير

العنصر	معناه
Overall Result	تقييم عام لأمان آلية توليد التوكن (ممتاز / ضعيف...)
Effective Entropy	قياس العشوائية بال bits — كل ما زاد الرقم، كل ما كان صعب التنبؤ بالتوكن (مثال: 117 bit يعتبر قوي جداً)
Reliability	مستوى الثقة الإحصائية في النتيجة (Significance Level)
Sample	تفاصيل عن العينات اللي اتحللت (العدد، الطول...)

التحليل الإحصائي مش دليل قاطع 100%، لكنه مؤشر قوي جداً على مدى أمان آلية التوليد. موقع آمن هيدّيك نتيجة زي "10/10، مفيش ثغرة واضحة" في التقييم العام.

11 Logger — سجلّ كل حركة مرور

الفكرة

Logger هو تاب ينفّص كل طلب مرّ من خلال Burp بتفاصيل أدق من HTTP History العادي — يقدر يوريك أعمدة إضافية زي ال Port، ال IP المستخدم، ال URL الكامل، وغيرها، وتقدر تفلتر وترتبّ على أساسها.

الفائدة العملية الأساسية ليه إنك تستخدمه كـ "صندوق أسود" ترجعه لما حاجة ماشتغلتش زي ما توقعت — مثلاً لو عملت Payload وانحجب (اتقفش) بواسطة Web Application Firewall، ترجع لل Logger وتحلل بالظبط إيه اللي اتبعت وإيه اللي رجع، عشان تفهم إزاي ال WAF قدر يكتشفه وتعّدل استراتيجيتك.

Organizer 12 — أرشفة الطلبات

الفكرة

مجرد "دفتر ملاحظات" منظم جوه Burp — بتحفظ فيه نسخ (Read-only) من طلبات مهمة عايز ترجعلها بعدين، أو تحطها في تقرير الاختراق (Pentest Report).

• بتبعتله طلبات بـ **Send to Organizer** أو **Ctrl + 0**.

• كل طلب محفوظ هو نسخة ثابتة (Read-only) من لحظة الحفظ، معاها معلومات زي: Status Code، HTTP Method، Host، عدد ال Parameters، وأي ملاحظات تضيفها.

استخدمه كـ "سلة" لحفظ الأدلة (Evidence) أثناء الاختبار بدل ما تعتمد على الذاكرة أو تاخذ Screenshots متفرقة.

Extensions 13 — توسعة قدرات Burp

الفكرة Burp يوفر API مفتوح يخلي المطورين يضيفوا وظائف جديدة كلياً للأداة — من خلال BApp Store (متجر الإضافات الرسمي) أو إضافات مكتوبة يدوياً.

واجهة Extensions

- **Extensions List**: كل الإضافات المثبتة، تقدر تفعل/تعطل كل واحدة.
- **Add / Remove**: تثبيت إضافة من ملف على الجهاز، أو إزالة إضافة.
- **Up/Down**: ترتيب تشغيل الإضافات — ييتنفذوا بترتيب تنازلي من فوق لتحت، ومهم جداً لما يكون فيه إضافتين يبعدلوا نفس الطلب (ممكن يتعارضوا).
- **Details / Output / Errors**: معلومات ومخرجات وأخطاء الإضافة المختارة.

لغات البرمجة المدعومة

اللغة	ملاحظات
Java	متكاملة مع Burp بشكل مباشر (Native)
Python	تحتاج تثبيت Jython Interpreter (تنزل ال Standalone JAR وتحدد مساره في Extensions Settings)
Ruby	تحتاج JRuby بنفس فكرة Jython

مثال على إضافة مفيدة: Request Timer

إضافة من BApp Store بتقيس الوقت اللي محتاجه كل Request عشان يوصله Response — مفيدة جداً لاكتشاف ثغرات زي **Time-based attacks**، مثلاً لو فورم اللوجن بياخذ وقت أطول شوية مع يوزرات

موجودة فعلاً عن اليوزرات الغير موجودة، ده بيكشفلك "Username Enumeration" عن طريق الفرق في زمن الاستجابة.

14 جدول مقارنة شامل بين كل الأدوات

الأداة	الوظيفة الأساسية	تلقائي ولا يدوي؟	متى تستخدمها
Dashboard	نظرة عامة على المهام والثغرات المكتشفة تلقائياً	تلقائي	بداية كل جلسة اختبار
Target	بناء خريطة الموقع وتحديد نطاق الاستهداف	شبه تلقائي	تنظيم وفلتر ما قبل بدء الاختبار
Proxy	اعتراض الحركة بين المتصفح والسيرفر	شبه يدوي	الخطوة الأولى دائماً – تجميع الطلبات
Repeater	تعديل وإعادة إرسال طلب واحد بشكل مررّ	يدوي 100%	تحليل عميق لثغرة معينة (SQLi, IDOR, Logic flaws)
Intruder	إرسال نفس الطلب بتعديلات متعددة آلياً	شبه آلي (Fuzzing)	Brute-force, Credential Stuffing, اكتشاف Endpoints
Decoder	ترميز/فك ترميز/تجزئة البيانات	يدوي	فهم بيانات مشفرة، توليد Payloads، فحص Hashes
Comparer	مقارنة نصّين أو ملفين	يدوي	اكتشاف الفروق الدقيقة بين Responses متشابهة
Sequencer	تحليل عشوائية التوكات	آلي (تجميع) + تحليل	اختبار أمان Session/CSRF Tokens
Logger	سجل تفصيلي لكل الحركة	تلقائي	تحليل عميق لما يحصل من غير ما نتوقعه
Organizer	أرشفة الطلبات المهمة	يدوي	توثيق الأدلة للتقرير النهائي

تخصيص Burp لاحتياجات خاصة	حسب الإضافة	إضافة وظائف جديدة للأداة	Extensions
---------------------------	-------------	--------------------------	------------

إزاي الأدوات دي بتترابط مع بعض فعلياً؟

السيناريو ده بيوضح إزاي محترف ال Pentesting يستخدم الأدوات دي مع بعض في اختبار حقيقي لتطبيق ويب:

1. **Target + Proxy** → تصفح التطبيق بشكل طبيعي مع تفعيل Intercept أحياناً، وحدد ال Scope بدري عشان تبعد عن الضوضاء، عشان تجمع أكبر عدد ممكن من ال Endpoints وتفهم بنية الموقع (Mapping the Attack Surface).

2. **Repeater** → لما تلاقي Endpoint مشبوه (زي `/about/2` أو `/products/3`)، تبعته ل Repeater وتبدأ تختبره يدوياً: تضيف علامة اقتباس، تغير Headers، تجرب SQLi خطوة بخطوة وأنت بتحلل كل Response.

3. **Decoder** → لو لقيت بيانات غريبة (Cookie مشفر، Parameter base64، Token)، تاخدها ل Decoder تفك تشفيرها أو تحلل تركيبها قبل ما تستخدمها في هجوميك.

4. **Intruder** → بعد ما تأكدت إن فيه Vulnerability (زي صفحة لوجن من غير Rate Limiting)، تستخدم Intruder عشان تؤتمت الهجوم — Sniper ل fuzzing endpoint، أو Pitchfork ل credential stuffing بلسته يوزر/باسورد مسربة.

5. **Comparer** → لما نتائج Intruder كلها بنفس ال Status Code، تستخدم Comparer تقارن Response ناجح محتمل مع Response فاشل عادي عشان تلاقي الفرق اللي ياكذلك نجاح الهجوم.

6. **Sequencer** → لو الهدف كان اختبار قوة ال Session Tokens نفسها (مش محتواها) أو اكتشاف Session Fixation، تستخدم Sequencer يحلل مدى عشوائيتها إحصائياً.

7. **Logger + Organizer** → ترجع لل Logger لو حاجة ماشتغلتش كما توقعت عشان تحلل السبب، وكل خطوة مهمة وصلت لنتيجة، تحفظها في Organizer كدليل (Evidence) هتحتاجه وقت كتابة التقرير النهائي.

8. **Extensions** → لو احتجت قدرة مش موجودة أصلاً في Burp (زي قياس زمن الاستجابة بدقة لاكتشاف Username Enumeration، أو فحص خاص ب framework معين)، تدور في BApp Store على إضافة تغطي الاحتياج ده.

خلاصة الفكرة

Burp Suite مش مجموعة أدوات منفصلة عن بعض — هو منظومة واحدة متكاملة: Target يرسم الحدود، Proxy يجمع، Repeater يحلل يدوياً، Intruder بيؤتمت، Decoder و Comparer و Sequencer بيدعموا التحليل، Logger و Organizer بيوثقوا، و Extensions بتوسع كل ده. الفهم الحقيقي مش إنك تحفظ كل زرار بيعمل إيه، لكن إنك تفهم "امتى أستخدم إيه" وإزاي الأدوات دي بتكمل بعض في ال workflow الكامل بتاع اختبار الاختراق.

مصادر إضافية للتعلم: (academy.portswigger.net) PortSwigger Web Security Academy —
فيه معمل كامل لكل نوع ثغرة بتكلم عنها هنا، ومجاني بالكامل.